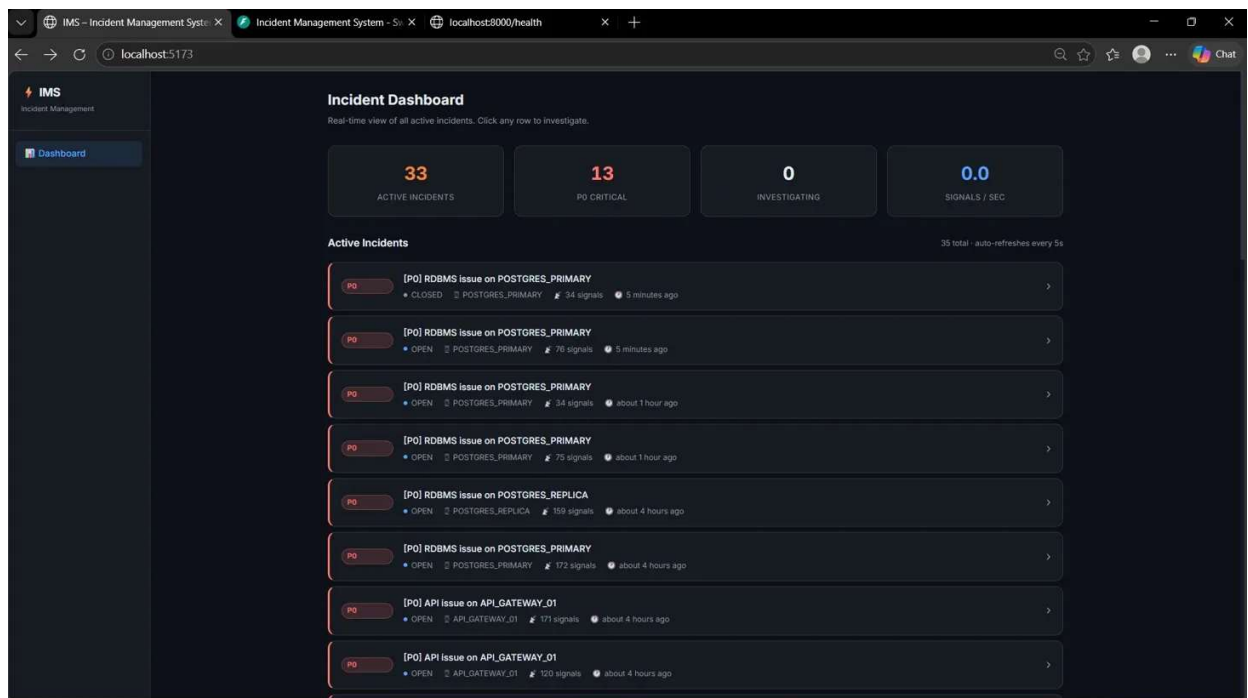


Incident Management System

Product Demonstration -End-to-End Incident Lifecycle.

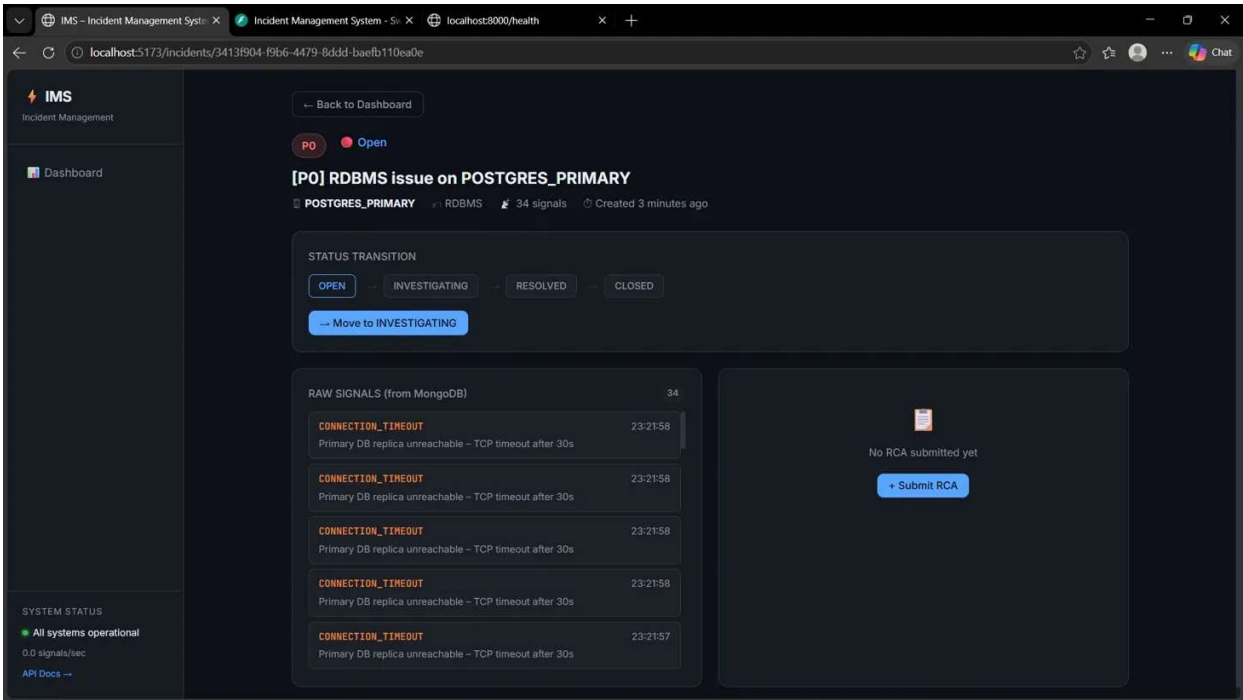
The Incident Management System is a production-grade, full-stack platform built to monitor distributed infrastructure and manage failure workflows end to end. It ingests high-volume monitoring signals from databases, caches, API gateways, and message queues — handling burst traffic using an in-memory ring buffer and async worker pipeline, ensuring the system never crashes under load. A debounce engine groups repeated signals from the same component into a single incident, preventing alert noise. Each incident follows a strict state machine lifecycle from Open to Closed, with a mandatory Root Cause Analysis gate that blocks closure until engineers document the cause, fix, and prevention steps — with Mean Time To Repair calculated automatically. The backend is built with FastAPI and Python, backed by PostgreSQL, MongoDB, and Redis, while the React frontend provides a live incident feed, signal viewer, and RCA form — all auto-refreshing in real time. Below are the screenshots of demonstration.

1- Incident Dashboard (Live Feed)



The main command centre of the IMS. Displays real-time metrics showing 33 active incidents, 13 P0 critical alerts, and live signals per second. All incidents are listed in priority order with component name, signal count, status, and age — auto-refreshing every 5 seconds.

2- Incident Detail View (Signal Audit Log):



The drill-down page for a single P0 incident on POSTGRES_PRIMARY. Shows the full status transition pipeline (OPEN → INVESTIGATING → RESOLVED → CLOSED), and pulls all 34 raw signals directly from MongoDB in real time, proving the audit log is live and query able.

3- Root Cause Analysis Form:

The screenshot displays the Incident Management System (IMS) interface. The browser tabs show 'IMS - Incident Management System' and 'localhost:8000/health'. The address bar shows the incident URL: 'localhost:5173/incidents/3413f904-f9b6-4479-8ddd-baefb110ea0e'. The incident title is 'POSTUNCO_PRIMARY'. The status is 'RESOLVED'. The left sidebar shows 'SYSTEM STATUS' as 'All systems operational' with '0.0 signals/sec' and a link to 'API Docs'. The main area shows 'RAW SIGNALS (from MongoDB)' with 34 signals, all of type 'CONNECTION_TIMEOUT' with the message 'Primary DB replica unreachable - TCP timeout after 30s'. The right panel contains the 'Root Cause Analysis' form with fields for 'INCIDENT START' (2026-05-05 00:00), 'INCIDENT END' (2026-05-14 00:00), 'ROOT CAUSE CATEGORY' (Hardware Failure), 'FIX APPLIED' (Fixed the component), and 'PREVENTION STEPS' (Regular maintenance required). A 'Submit RCA' button is at the bottom.

STATUS TRANSITION

OPEN -- INVESTIGATING -- **RESOLVED** -- CLOSED

Move to CLOSED ⚠ RCA required before closing

RAW SIGNALS (from MongoDB) 34

SIGNAL	TIME
CONNECTION_TIMEOUT Primary DB replica unreachable - TCP timeout after 30s	23:21:58
CONNECTION_TIMEOUT Primary DB replica unreachable - TCP timeout after 30s	23:21:58
CONNECTION_TIMEOUT Primary DB replica unreachable - TCP timeout after 30s	23:21:58
CONNECTION_TIMEOUT Primary DB replica unreachable - TCP timeout after 30s	23:21:58
CONNECTION_TIMEOUT Primary DB replica unreachable - TCP timeout after 30s	23:21:57

SYSTEM STATUS

All systems operational

0.0 signals/sec

API Docs →

Root Cause Analysis

INCIDENT START * 2026-05-05 00:00

INCIDENT END * 2026-05-14 00:00

ROOT CAUSE CATEGORY * Hardware Failure

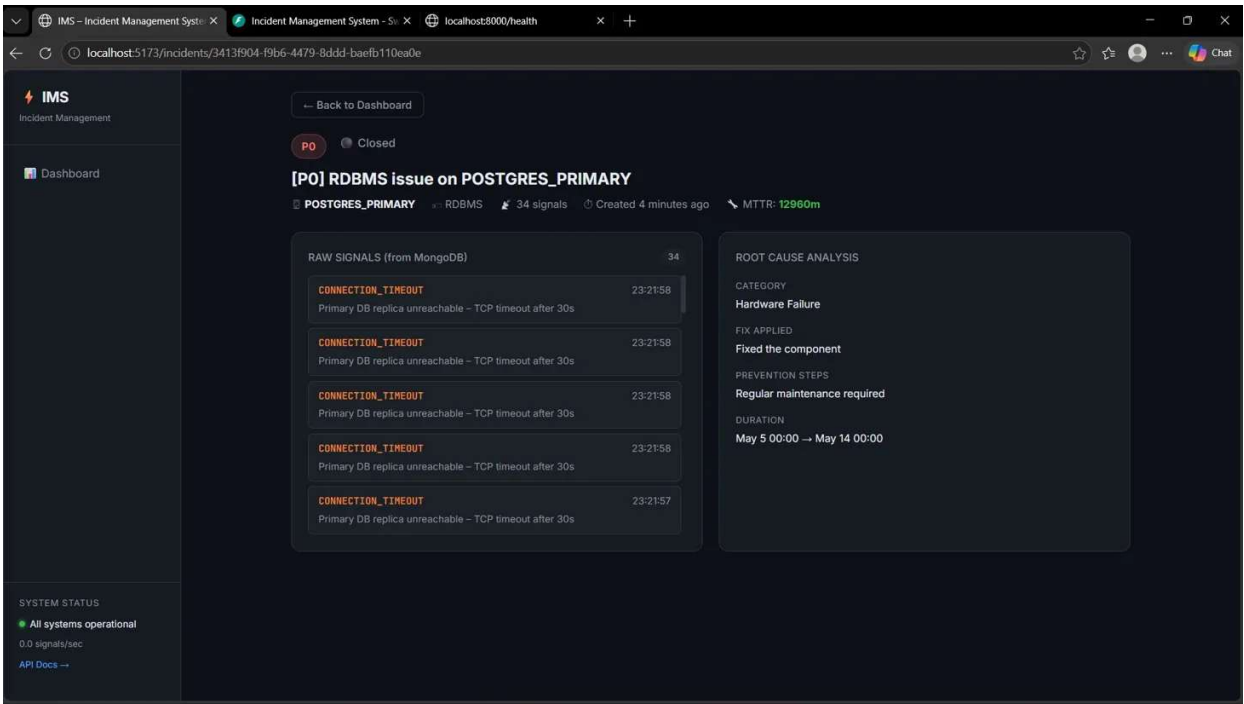
FIX APPLIED * Fixed the component

PREVENTION STEPS * Regular maintenance required

Submit RCA

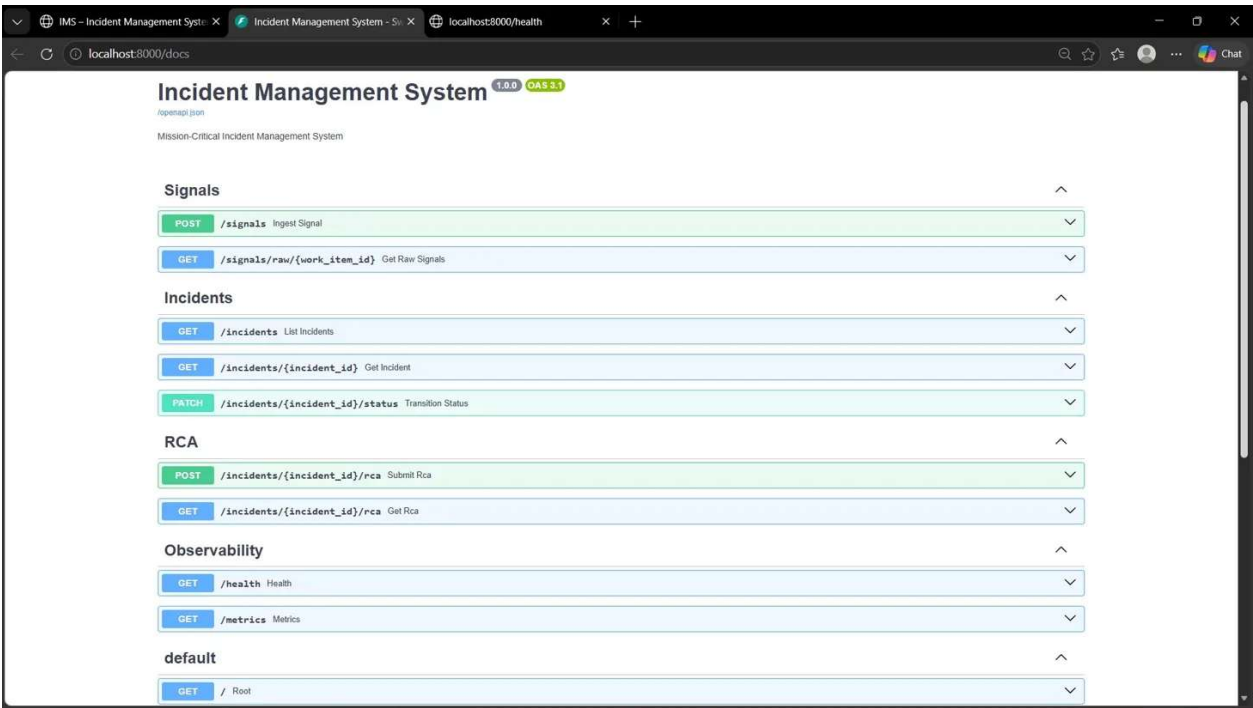
The mandatory RCA form that engineers must complete before an incident can be closed. Captures incident start and end timestamps, root cause category (Hardware Failure), fix applied, and prevention steps — all validated before submission is accepted.

4- Closed Incident with MTTR:



A fully resolved and closed incident showing the completed RCA details alongside the automatically calculated MTTR of 12,960 minutes. Demonstrates the full lifecycle — from open alert to closed incident with documented root cause, fix, and prevention steps.

5- API Documentation:



The interactive API documentation at localhost:8000/docs, showing all 9 endpoints grouped by function: Signals ingestion, Incidents management, RCA submission, and Observability (health + metrics). Every endpoint is live-testable directly from the browser.

6- Failure Simulation Script (Terminal):

```
PS C:\Users\hadhi\OneDrive\Desktop\ims> python scripts/seed_failure_event.py

🔥 Simulating: RDBMS Primary Outage (P0)
Sending 110 signals over ~8.8s

[ 10/110] ✓ Sent
[ 20/110] ✓ Sent
[ 30/110] ✓ Sent
[ 40/110] ✓ Sent
[ 50/110] ✓ Sent
[ 60/110] ✓ Sent
[ 70/110] ✓ Sent
[ 80/110] ✓ Sent
[ 90/110] ✓ Sent
[100/110] ✓ Sent
[110/110] ✓ Sent
✅ Sent 110/110 signals

🔥 Simulating: MCP Host Cascade (P1)
Sending 50 signals over ~7.5s

[ 10/50] ✓ Sent
[ 20/50] ✓ Sent
[ 30/50] ✓ Sent
[ 40/50] ✓ Sent
[ 50/50] ✓ Sent
✅ Sent 50/50 signals

🔥 Simulating: Cache Cluster Evictions (P2)
Sending 30 signals over ~6.0s

[ 10/30] ✓ Sent
[ 20/30] ✓ Sent
[ 30/30] ✓ Sent
✅ Sent 30/30 signals

Simulation complete! Check the dashboard at http://localhost:5173
```

The `seed_failure_event.py` script running in PowerShell, simulating a real cascading infrastructure failure across three severity levels. It sends 110 signals for a P0 RDBMS Primary Outage, 50 signals for a P1 MCP Host Cascade, and 30 signals for a P2 Cache Cluster Eviction — all 190 signals delivered successfully, triggering live incidents on the dashboard.

7- Load Test Results (Terminal):

```
PS C:\Users\hadhi\OneDrive\Desktop\ims> python scripts/load_test.py

Load Test: 200 signals/sec for 20s = 4,000 total
Target: http://localhost:8000

Backend: ok

[ 20.1s] Sent: 3,040 | OK: 3,040 | 429/err: 0 | Rate: 151/sec

Results:
Duration: 20.1s
Total sent: 3,040
Successful: 3,040
Rate limited: 0
Actual rate: 151/sec
```

The `load_test.py` script demonstrating the system's high-throughput ingestion capability. It sent 3,040 signals at 151 signals/sec over 20 seconds with a 100% success rate — zero rate-limited, zero errors — proving the ring buffer and async worker pipeline handle burst traffic without any data loss or system crash.